

Sprint 5 · Matemáticas y estadística en código

S5.1 · NumPy y álgebra lineal

Domina las herramientas numéricas esenciales de Python: desde arrays eficientes hasta operaciones de álgebra lineal aplicada. Este sprint te prepara para trabajar con datos a escala real.

$$\int_0^{\infty} e^{-x^2} dx = \sqrt{\frac{\pi}{2}}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$E = mc^2$$

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u$$

$$\sum_m^n n = \frac{11}{\infty} \frac{1^2}{n} = \frac{\pi^2}{6}$$

$$i\hbar \partial\varphi/\partial t = \bar{H}_H\varphi$$

$$a^2 + b^2 = c^2$$

Índice de contenidos

¿Qué aprenderemos hoy?

01

Por qué NumPy

La base de la computación científica en Python

03

Indexación y slicing

Acceso flexible y preciso a los datos

05

Álgebra lineal

Dot, matmul, inversas y autovalores

02

Arrays y tipos de dato

Estructura fundamental y eficiencia de memoria

04

Vectorización y broadcasting

Operaciones masivas sin bucles

06

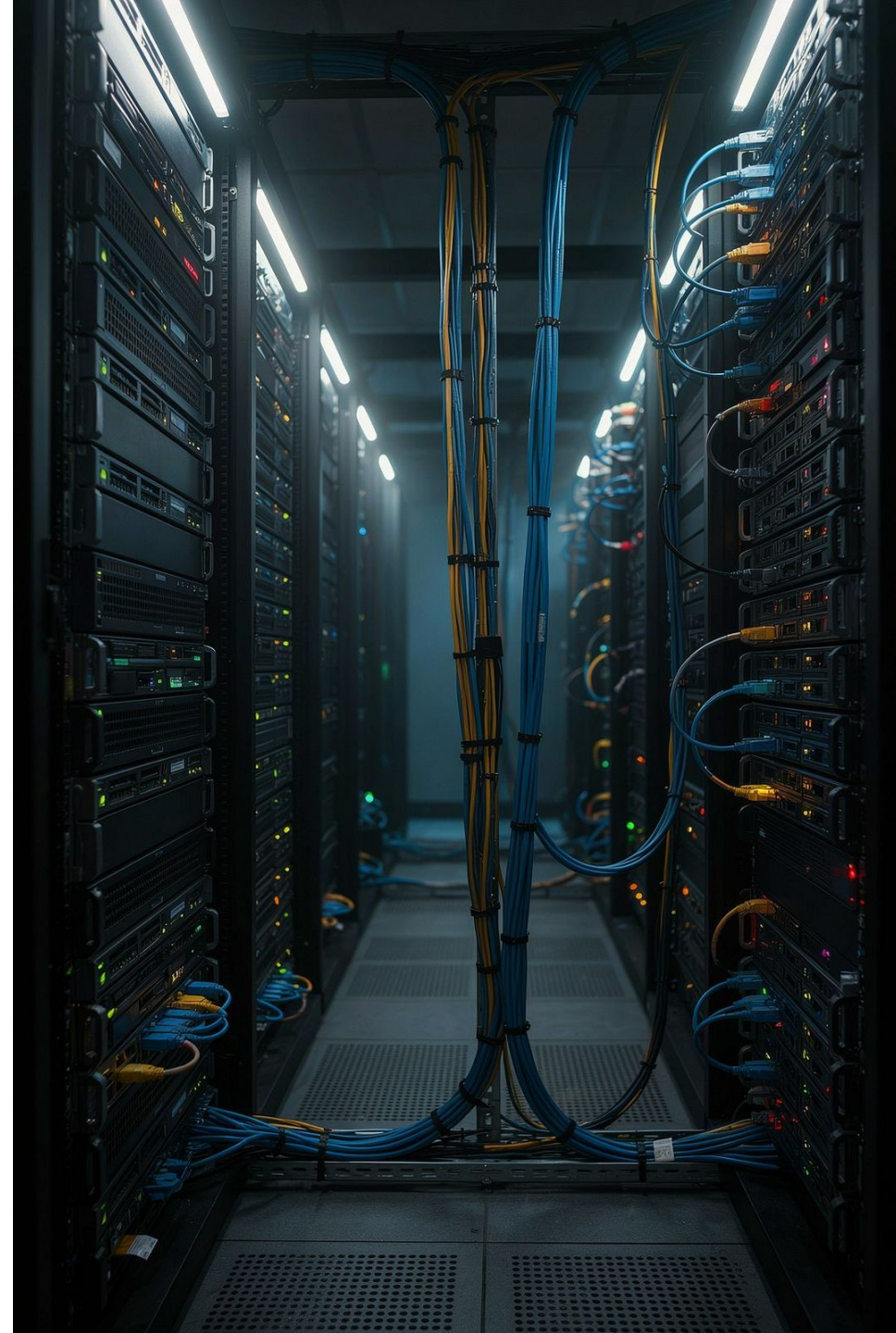
Aleatoriedad y semillas

Reproducibilidad en experimentos numéricos

Módulo 1

¿Por qué NumPy?

Python puro es flexible, pero lento para operar sobre grandes volúmenes de datos numéricos. NumPy resuelve este problema ofreciendo arrays contiguos en memoria y rutinas compiladas en C que superan en velocidad a los bucles Python por órdenes de magnitud.



Python puro vs. NumPy

❌ Lista de Python

```
data = list(range(1_000_000))
total = 0
for x in data:
    total += x * 2
```

Cada elemento es un objeto Python completo. El bucle interpreta instrucción por instrucción. Lento y costoso en memoria.

✅ Array de NumPy

```
import numpy as np
data = np.arange(1_000_000)
total = (data * 2).sum()
```

Los datos se almacenan como valores contiguos en memoria. Las operaciones se ejecutan en C vectorizado. Hasta **100x más rápido**.

📌 NumPy es la piedra angular de pandas, scikit-learn, TensorFlow y prácticamente todo el ecosistema científico de Python.

Módulo 2

Arrays y tipos de dato

El objeto central de NumPy es el `ndarray`: una rejilla N-dimensional de elementos del mismo tipo. A diferencia de las listas Python, todos los elementos comparten un único tipo (`dtype`), lo que permite un almacenamiento compacto y operaciones ultrarrápidas.

Enteros

`int8`, `int16`, `int32`, `int64` — para números enteros con distinta precisión y rango.

Flotantes

`float32`, `float64` — el más usado en ciencia de datos y deep learning.

Complejos

`complex64`, `complex128` — para análisis de señales y transformadas de Fourier.

Booleanos

`bool` — resultado de operaciones lógicas; útil para máscaras y filtros.

Creación de arrays: recetario

```
import numpy as np

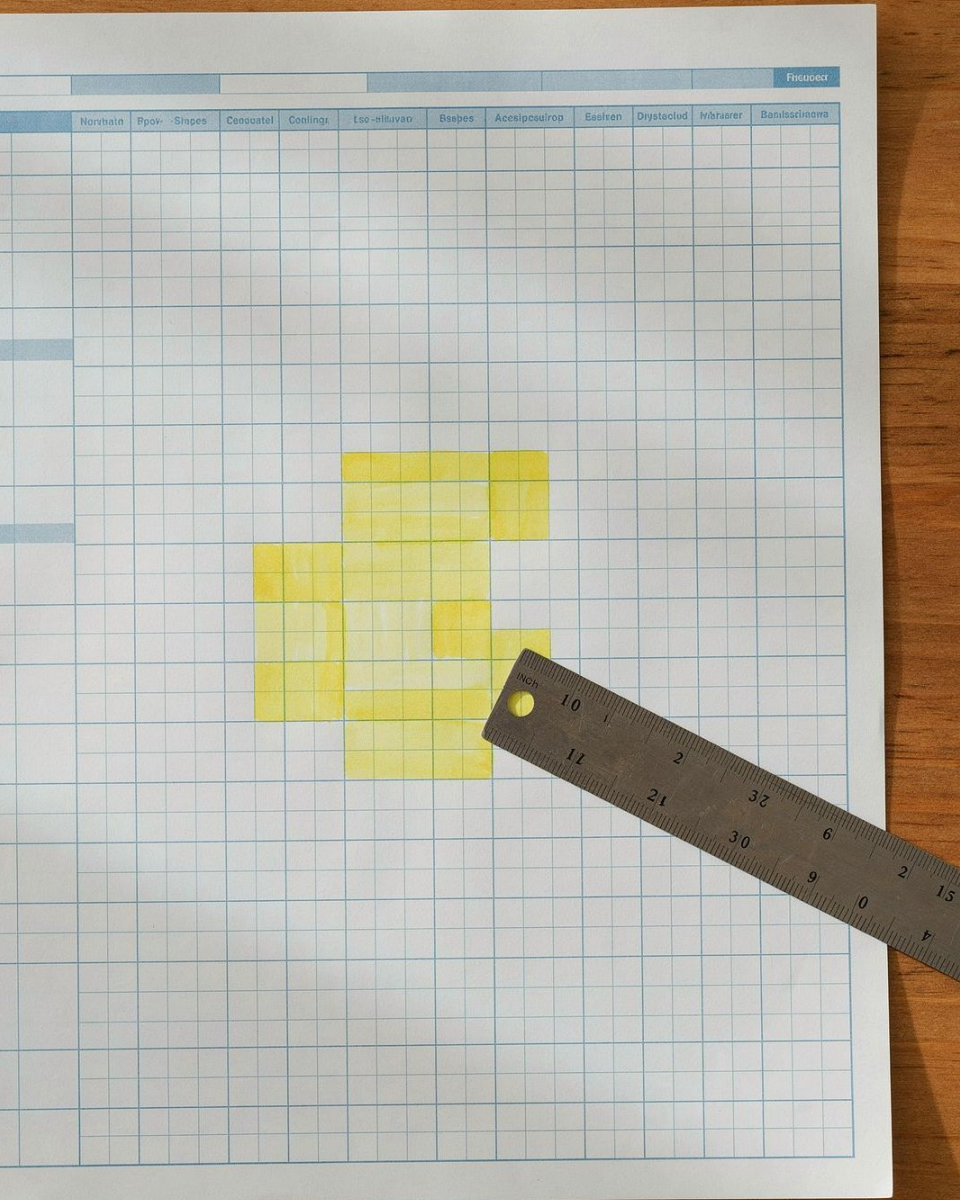
# Desde una lista
a = np.array([1, 2, 3, 4])    # dtype=int64 por defecto

# Rangos
b = np.arange(0, 10, 2)      # [0, 2, 4, 6, 8]
c = np.linspace(0, 1, 5)     # [0.0, 0.25, 0.5, 0.75, 1.0]

# Arrays especiales
zeros = np.zeros((3, 4))     # Matriz 3x4 de ceros
ones  = np.ones((2, 2), dtype=np.float32)
eye   = np.eye(3)            # Identidad 3x3
rand  = np.random.rand(3, 3) # Valores aleatorios [0,1)

# Atributos clave
print(a.shape, a.dtype, a.ndim, a.size)
```

Conocer los atributos `shape`, `dtype`, `ndim` y `size` es imprescindible para depurar errores de dimensiones en modelos y pipelines de datos.



Módulo 3

Indexación y slicing

NumPy ofrece formas muy expresivas de acceder a subconjuntos de datos: desde índices simples hasta selecciones booleanas avanzadas. Dominar el slicing elimina la necesidad de bucles para filtrar y transformar datos.

Tipos de indexación

Indexación básica y slicing

```
a = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

a[0, 1]    # 2 (fila 0, col 1)
a[1:, :2]  # [[4,5],[7,8]]
a[:, 2]    # [3, 6, 9] (col entera)
```

La sintaxis [inicio:fin:paso] funciona en cada dimensión de forma independiente.

Indexación booleana y fancy

```
mask = a > 5
a[mask]    # [6, 7, 8, 9]

# Fancy indexing
idx = np.array([0, 2])
a[idx, :]  # filas 0 y 2

# Modificar con máscara
a[a % 2 == 0] = 0 # pares → 0
```

Las máscaras booleanas son la forma idiomática de filtrar filas en DataFrames y arrays sin bucles.

Módulo 4

Vectorización frente a bucles



Vectorización: la vía rápida

Aplicar una operación sobre el array entero de una vez. NumPy delega el cálculo a rutinas BLAS/LAPACK compiladas. Sin overhead del intérprete Python por elemento.



Bucles explícitos: la vía lenta

Recorrer cada elemento con `for` en Python impone el coste del intérprete en cada iteración. Para arrays de millones de elementos, la diferencia es de segundos frente a minutos.

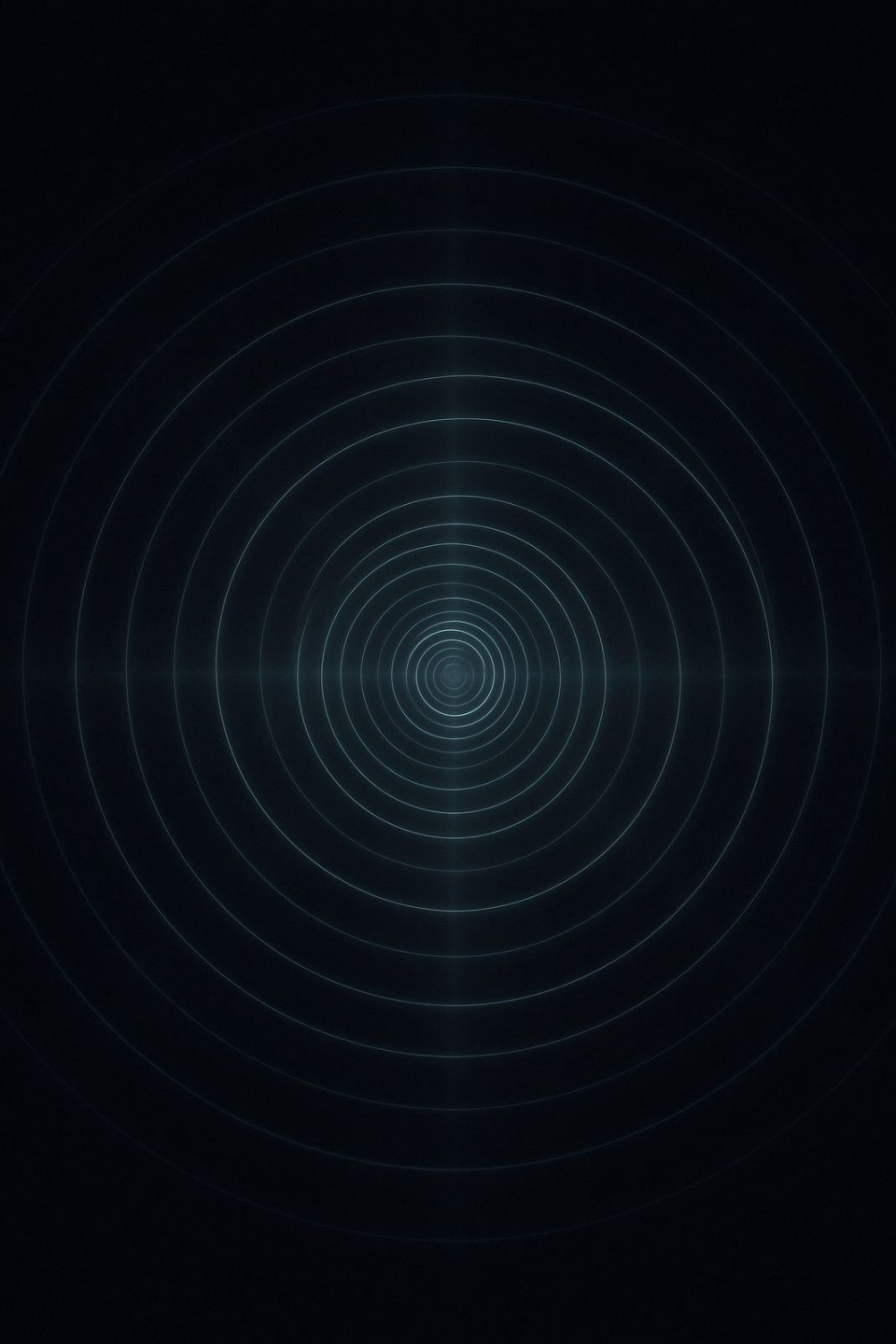
- ❏ Regla de oro: si ves un bucle `for` sobre un array NumPy, pregúntate si existe una operación vectorizada equivalente. Casi siempre la hay.

Vectorización en la práctica

```
import numpy as np
import time

n = 10_000_000
a = np.random.rand(n)

#
```

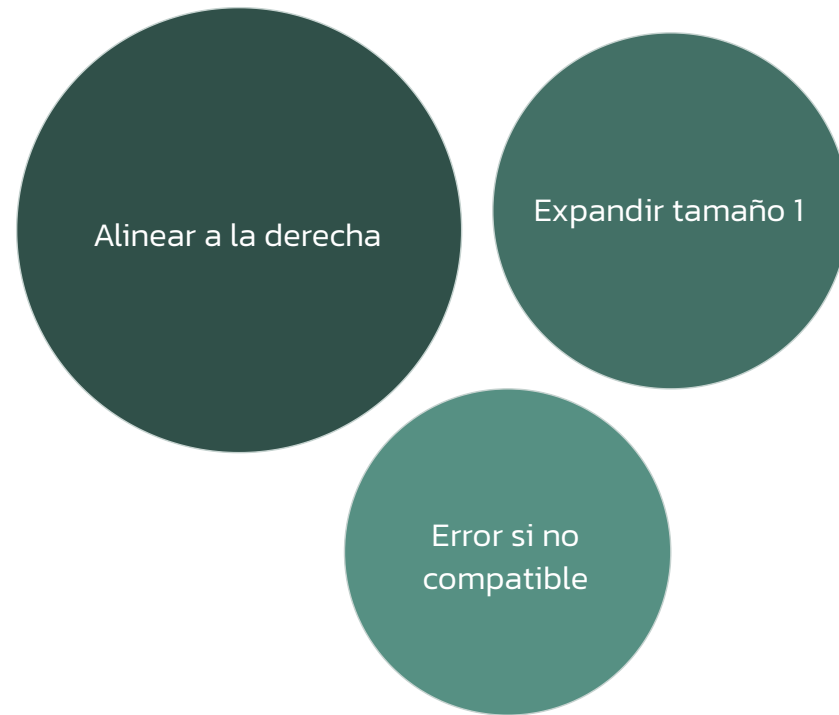
A decorative graphic on the left side of the slide, consisting of numerous concentric circles in a light blue/teal color, centered on a dark navy blue background. The circles vary in size, creating a tunnel-like or ripple effect.

Módulo 5

Broadcasting

El broadcasting permite que NumPy opere sobre arrays de formas distintas sin copiar datos. El motor expande automáticamente las dimensiones más pequeñas para que cuadren con las mayores, siguiendo un conjunto de reglas bien definidas.

Reglas del broadcasting



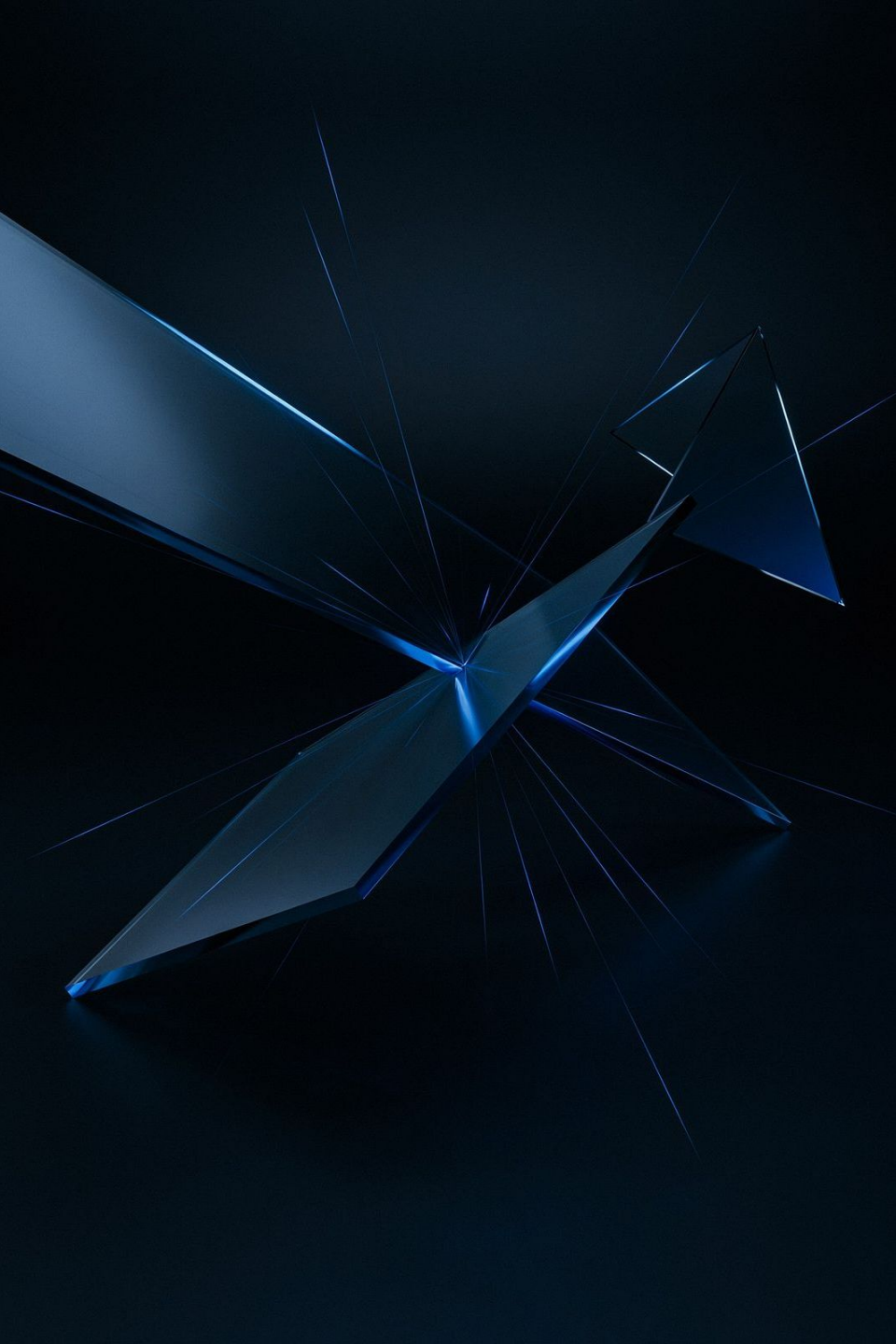
Ejemplo práctico

```
a = np.array([[1],[2],[3]]) # (3,1)
b = np.array([10, 20, 30]) # (3,)
# → broadcast a (3,3)
a + b
# [[11,21,31],
#  [12,22,32],
#  [13,23,33]]
```

Normalización de datos

```
X = np.random.rand(100, 5)
# Restar media de cada columna
X_norm = X - X.mean(axis=0)
# Dividir por desviación típica
X_std = X_norm / X.std(axis=0)
```

Broadcasting hace que la normalización de matrices sea una sola línea, sin bucles sobre columnas.



Módulo 6

Álgebra lineal con NumPy

El módulo `numpy.linalg` proporciona las operaciones matriciales que sustentan la regresión, las redes neuronales, el análisis de componentes principales (PCA) y muchos otros algoritmos de ciencia de datos.

Producto escalar y multiplicación matricial

`np.dot` — producto escalar

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.dot(a, b) # 1·4+2·5+3·6 = 32

# También funciona con matrices
A = np.random.rand(3, 4)
B = np.random.rand(4, 2)
C = np.dot(A, B) # (3,2)
```

`@` — operador matmul

```
A = np.random.rand(3, 4)
B = np.random.rand(4, 2)

# Equivalente moderno y legible
C = A @ B # (3,2)

# np.matmul vs np.dot:
# matmul NO acepta escalares
# dot SÍ acepta escalares y 1-D
# Prefiere @ para matrices
```

El operador `@` (PEP 465) es la forma recomendada en código moderno.

Inversa, determinante y sistemas lineales

```
from numpy import linalg as la

A = np.array([[2., 1.],
              [5., 3.]])

# Inversa
A_inv = la.inv(A)      # A @ A_inv ≈ I

# Determinante
det = la.det(A)        # 1.0

# Resolver sistema Ax = b (más eficiente que inv)
b = np.array([1., 2.])
x = la.solve(A, b)     # [1., -1.]

# Verificar
np.allclose(A @ x, b)  # True
```

⚠ Usar `la.solve` en lugar de `la.inv(A) @ b` es más estable numéricamente y entre 2-3× más rápido para matrices grandes.

Autovalores y descomposiciones

Autovalores y autovectores

```
A = np.array([[4, 2],  
              [1, 3]])  
  
vals, vecs = la.eig(A)  
# vals: [5., 2.]  
# vecs: matriz de autovectores (cols)  
  
# Para matrices simétricas  
# (covarianza, PCA) → eigh  
vals, vecs = la.eigh(A)
```

Los autovalores de la matriz de covarianza son la base del PCA.
`eigh` es más estable para matrices simétricas.

SVD — Descomposición en valores singulares

```
X = np.random.rand(5, 3)  
  
U, s, Vt = la.svd(X, full_matrices=False)  
#  $X \approx U @ \text{np.diag}(s) @ Vt$   
  
# Norma de Frobenius  
la.norm(X)  
  
# Rango numérico  
la.matrix_rank(X)
```

La SVD subyace en algoritmos de recomendación, compresión de imágenes y regresión Ridge.

Módulo 7

Aleatoriedad y semillas

Generar números aleatorios reproducibles es fundamental en ciencia de datos: permite comparar modelos de forma justa, depurar experimentos y publicar resultados verificables. NumPy ofrece una API moderna basada en generadores explícitos.



Generadores y distribuciones

```
import numpy as np

# API moderna (recomendada desde NumPy 1.17)
rng = np.random.default_rng(seed=42)

# Distribuciones
rng.random((3, 3))      # Uniforme [0, 1)
rng.integers(0, 10, size=5) # Enteros aleatorios
rng.normal(loc=0, scale=1, size=1000) # Gaussiana N(0,1)
rng.choice([1, 2, 3, 4], size=10, replace=True)

# Barajar y muestrear
data = np.arange(20)
rng.shuffle(data)      # In-place
sample = rng.choice(data, size=5, replace=False)

# API antigua (compatible pero menos recomendada)
np.random.seed(42)
np.random.randn(3, 3)
```

Fija siempre una semilla en tus notebooks para que los resultados sean **reproducibles** entre ejecuciones y entre compañeros de equipo.

Simulación de Monte Carlo

```
rng = np.random.default_rng(seed=0)
n = 1_000_000

# Estimar  $\pi$  lanzando puntos aleatorios en el cuadrado  $[-1,1]^2$ 
x = rng.uniform(-1, 1, n)
y = rng.uniform(-1, 1, n)

dentro = (x**2 + y**2) <= 1.0
pi_est = 4 * dentro.sum() / n

print(f" $\pi$  estimado: {pi_est:.4f}") #  $\approx 3.1416$ 
```

Este sencillo ejemplo muestra la potencia de combinar vectorización, broadcasting y aleatoriedad: un millón de puntos procesados en milisegundos. Las simulaciones Monte Carlo son la base de muchos métodos estadísticos y de optimización en ML.

Lo que has aprendido



Arrays

Tipos de dato, creación, atributos y estructura N-dimensional del `ndarray`.



Indexación

Slicing, máscaras booleanas y fancy indexing para acceso eficiente.



Vectorización

Ufuncs y operaciones vectorizadas que superan a los bucles Python por órdenes de magnitud.



Broadcasting

Operar sobre arrays de distinta forma sin copiar memoria ni usar bucles.



Álgebra lineal

Dot, matmul, inversa, `solve`, autovalores y SVD con `numpy.linalg`.



Aleatoriedad

Generadores reproducibles con semilla y las distribuciones más habituales.